LAB 5: STATIC CODE ANALYSIS

Flight Reservation System | Final Report

Student	Pradhyumn Anand	**SRN**	PES2UG24CS358
Python file	flight_reservation_system.py	**GitHub branch**	lab5-static-analysis
Repository	github.com/pradhyumnanand01/L5_50_flight_reservation_system/tree/lab5-static-analysis	**Commit**	c090c6c

1. Issue Table

Issue	Type	Original line(s)	Description	Fix approach
Unsafe eval()	Security	47	Evaluated a string as Python code, creating risk of arbitrary expression execution.	Replaced with a validated numeric surcharge calculation.
Shell command execution	Security	63	Used os.system() with a command assembled using a filename.	Removed shell execution and used Python file operations.
Hardcoded secret	Security	75	A token-like value was embedded in source and printed.	Removed hardcoded token and its output.
Weak MD5 hash	Security	52	MD5 was used to hash a passenger identifier.	Changed digest to SHA-256; hashing is not a substitute for protecting personal data.
Predictable PIN	Security	56	Used a non-cryptographic random generator for a PIN.	Changed to secrets.randbelow().
Mutable default	Bug / quality	15	A list default could be shared between calls.	Used None and initialized a fresh list inside the function.
Bare exception	Reliability	31	Broad exception handling silently hid errors.	Removed broad exception suppression.
File handling	Reliability	60–63	File was opened without context-manager cleanup or explicit encoding.	Used Path.open() in a with block and UTF-8 encoding.

Line numbers refer to the original source/report and can change after editing.

2. Reflection Questions and Answers

The provided handout did not include the exact five question texts. These are suggested reflection questions; replace them if your instructor supplied different questions.

1. What is static code analysis, and why is it useful?

Static code analysis examines source code without running the program. It helps identify potential bugs, insecure coding patterns, and style problems early, making code easier to maintain and review.

2. What are the differences between Pylint, Bandit, and Flake8?

Pylint checks general code quality, naming, documentation, and design warnings. Bandit looks for common security risks in Python code. Flake8 checks style and other code issues, including PEP 8 formatting violations.

3. What important issues were identified in the original program?

The original reports identified unsafe eval() use, shell execution through os.system(), a hardcoded token, weak MD5 hashing, predictable random PIN generation, a mutable default list, a bare exception handler, and unsafe file handling.

4. How were the issues addressed?

Expression evaluation was replaced with a validated numeric calculation; shell execution and the hardcoded token were removed; SHA-256 replaced MD5; PIN generation now uses secrets; the mutable default became None; and file handling was improved.

5. What did you learn, and what are the limitations of static analysis?

Static analysis can find many problems before execution, but clean reports do not prove software is completely secure or functionally correct. Functional tests and code review are also needed.

3. Final Verification Results

Tool	Observed result
Pylint	10.00/10 (reported in terminal on 15 October 2026).
Bandit	No issues identified; all severity counts were zero.
Flake8	Output file was empty in the supplied terminal output, consistent with no findings; confirm with a direct run if required.
GitHub	Branch pushed successfully; latest commit shown in supplied terminal output: c090c6c.

4. Edited Python Code

Appendix A — cleaned version of flight_reservation_system.py. Verify this appendix matches the final file in the GitHub branch before submission.

```
"""Flight reservation system for the Lab 5 static analysis exercise."""
import hashlib
import logging
import secrets
from pathlib import Path
logging.basicConfig(
    level=logging.INFO,
    format="%(levelname)s: %(message)s",
)
flights = {
    "AI101": {
        "destination": "London",
        "seats": 2,
        "fare": 450.0,
        "is_full": False,
    },
    "BA202": {
        "destination": "New York",
        "seats": 0,
        "fare": 600.0,
        "is_full": True,
    },
    "EK303": {
        "destination": "Dubai",
        "seats": 5,
        "fare": 320.0,
        "is_full": False,
    },
}
def book_flight_seat(
    flight_no, passenger, luggage_weight=15, baggage_tags=None
):
    """Book one seat and record the booking."""
    if baggage_tags is None:
        baggage_tags = []
    if flight_no not in flights:
        logging.warning("Unknown flight: %s", flight_no)
        return False
    flight = flights[flight_no]
    if flight["is_full"] or flight["seats"] &lt;= 0:
        logging.info("No seats available on %s", flight_no)
        return False
    if not isinstance(passenger, str) or not passenger.strip():
        raise ValueError("Passenger name must not be empty.")
    if (
        isinstance(luggage_weight, bool)
        or not isinstance(luggage_weight, (int, float))
        or luggage_weight &lt; 0
    ):
        raise ValueError("Luggage weight must be non-negative.")
    flight["seats"] -= 1
    flight["is_full"] = flight["seats"] == 0
    baggage_tags.append(
        f"Flight {flight_no}: {passenger}, "
        f"baggage weight {luggage_weight}"
    )
    logging.info("Seat booked on %s for %s", flight_no, passenger)
    return True
def cancel_booking(flight_no):
    """Restore a seat on a known flight."""
    if flight_no not in flights:
        logging.warning("Unknown flight: %s", flight_no)
        return False
    flight = flights[flight_no]
    flight["seats"] += 1
```

```
    flight["is_full"] = False
    logging.info("Seat restored on %s", flight_no)
    return True
def check_seat_availability(flight_no):
    """Return whether a flight has an available seat."""
    flight = flights.get(flight_no)
    if flight is None:
        return False
    return not flight["is_full"] and flight["seats"] &gt; 0
def calculate_baggage_fee(flight_no, surcharge_rate=0.15):
    """Calculate baggage surcharge using a validated rate."""
    if flight_no not in flights:
        raise ValueError(f"Unknown flight: {flight_no}")
    if (
        isinstance(surcharge_rate, bool)
        or not isinstance(surcharge_rate, (int, float))
        or not 0 &lt;= surcharge_rate &lt;= 1
    ):
        raise ValueError("Surcharge rate must be between 0 and 1.")
    base_fare = flights[flight_no]["fare"]
    return base_fare * surcharge_rate
def generate_passenger_hash(passenger_id):
    """Return a SHA-256 digest of a passenger identifier."""
    if not isinstance(passenger_id, str) or not passenger_id:
        raise ValueError("Passenger identifier must not be empty.")
    return hashlib.sha256(passenger_id.encode("utf-8")).hexdigest()
def generate_boarding_pin():
    """Generate a four-digit boarding PIN."""
    return secrets.randbelow(9000) + 1000
def export_flight_manifest(filename="flight_manifest.txt"):
    """Write the flight manifest using safe file handling."""
    manifest_path = Path(filename)
    with manifest_path.open("w", encoding="utf-8") as manifest_file:
        manifest_file.write(str(flights))
    logging.info("Flight manifest saved to %s", manifest_path)
def get_flight_summary(flight_data):
    """Count flights that have available seats."""
    return sum(
        1
        for details in flight_data.values()
        if not details["is_full"] and details["seats"] &gt; 0
    )
def main():
    """Run a demonstration of the flight reservation system."""
    book_flight_seat("AI101", "Kritika", 20)
    cancel_booking("BA202")
    print("Is AI101 open:", check_seat_availability("AI101"))
    print("Baggage Fee:", calculate_baggage_fee("AI101"))
    print(
        "Passenger Hash:",
        generate_passenger_hash("PASSPORT_IND_119"),
    )
    print("Boarding PIN:", generate_boarding_pin())
    export_flight_manifest()
    print("Available flights:", get_flight_summary(flights))
if __name__ == "__main__":
    main()
```